Vietnam National University - Ho Chi Minh City
University of Science
Faculty of Information Technology

SEMINAR DOCUMENT

Team Members

Name	ID	Email
Cao Uyển Nhi	22127310	cunhi22@clc.fitus.edu.vn
Lưu Thanh Thuý	22127410	ltthuy22@clc.fitus.edu.vn
Nguyễn Phước Minh Trí	22127424	npmtri22@clc.fitus.edu.vn
Võ Lê Việt Tú	22127435	vlvtu22@clc.fitus.edu.vn
Trần Thị Cát Tường	22127444	ttctuong22@clc.fitus.edu.vn

Supervisors

Name
Teacher Trần Duy Hoàng
Teacher Hồ Tuấn Thanh
Teacher Trương Phước Lộc

June 16, 2025

Contents

Revision History

Version	Date	Author	Description
1.0	2025-06-10	Cao Uyển Nhi	Initial template creation and project structure setup
1.1	2025-06-12	Cao Uyển Nhi	Added content structure, sections, and project planning
1.2	2025-06-14	Trần Thị Cát Tường	Added JMeter architecture and components research
1.3	2025-06-14	Lưu Thanh Thuý	Added basic performance testing concepts and methodology
1.4	2025-06-16	Nguyễn Phước Minh Trí	Completed performance testing tools comparison research
1.5	2025-06-17	Võ Lê Việt Tú	Added JMeter installation guide and environment setup documentation
1.6	2025-06-19	Cao Uyển Nhi	Added AI applications in performance testing research
1.7	2025-06-19	Lưu Thanh Thuý	Completed business flow analysis for JPetStore system
1.8	2025-06-20	Trần Thị Cát Tường	Designed test scenarios and test plan for JMeter implementation
1.9	2025-06-21	Cao Uyển Nhi	Consolidated research findings and prepared final report structure

Chapter 1

Introduction

1.1 Group Information

The team consists of 5 members with clearly defined roles to ensure schedule adherence and research quality:

Role	Member	Primary Responsibilities
Team Lead	Cao Nhi	Coordination, AI research in Performance Testing, consolidation & presentation
Member	Thanh Thuý	Foundational research & business flow analysis
Member	Minh Trí	Tool comparison, scenario design & JMeter scripting
Member	Cát Tường	In-depth JMeter study (architecture, components) & test plan design
Member	Việt Tú	Environment setup, test execution, demo preparation

1.2 Overview

1.2.1 Motivation

Performance is a critical factor shaping the user experience of any web application. A system may provide a rich interface and complete functionality, but if it responds slowly or fails under heavy traffic, users will quickly abandon it. In practice, poor performance has often led to revenue loss, reputational damage, and difficulty in scaling operations.

For this reason, Performance Testing is considered an essential step in the software development lifecycle. At the same time, the recent progress in Artificial Intelligence (AI) opens up new opportunities: faster analysis of large volumes of test data, automated anomaly detection, and improved accuracy in diagnosing performance issues. These aspects motivated us to pursue the topic "Performance Testing with Apache JMeter and AI-assisted Analysis."

1.2.2 Research Problem

The study focuses on addressing three key questions:

- How can we select the most suitable testing tool among the many available solutions?
- What is the proper way to design scenarios that closely reflect real user behavior?
- At which stages of the Performance Testing cycle can AI provide meaningful support?

1.2.3 Objectives

1. Standardize the concepts and scope of Performance Testing, with emphasis on Load, Stress, Spike, and Endurance tests.
2. Compare major performance testing tools (JMeter, NeoLoad, WebLOAD, LoadUI, LoadRunner) in terms of strengths, limitations, and usage contexts.
3. Gain practical knowledge of Apache JMeter, including its architecture, components, and scripting process.
4. Explore the potential of AI techniques (e.g., anomaly detection, log/result analysis).
5. Build and execute a complete testing scenario on JPetStore (Register → Login → Purchase → Payment).
6. Collect results, perform analysis, and deliver findings in the form of a written report, presentation, and demonstration.

1.2.4 Scope

1. **In scope**

- Theoretical study of Performance Testing types (Load, Stress, Spike, Endurance).
- Tool comparison at an overview level with key technical criteria.
- Experiments on a demo environment (not production).
- Proof-of-concept (POC) exploration of AI in result analysis.

2. **Out of scope**

- Advanced topics such as Security, Penetration, or Mobile Performance Testing.
- Development of new testing tools from scratch.
- Optimization of real production infrastructure.

1.2.5 Project Roadmap

Phase	Weeks	Main Activities	Status
Phase 1: Foundation & Research	1–3	Theoretical study, tool comparison, JMeter setup	Completed
Phase 2: Design & Execution	4–7	Scenario design, scripting, test execution, AI trials	Upcoming
Phase 3: Reporting & Presentation	8–9	Final report, slides, demo video	Pending

1.2.6 Stakeholders

Type	Stakeholder	Expectations
Academic	Supervisor	Quality of research and practical contribution
Students	Class peers	Reference for methodology and results
Community	Testers / Researchers	Insights on AI application in Performance Testing

1.2.7 Risks and Mitigation

Risk	Impact	Mitigation
Insufficient machine resources under high load	Distorted measurements	Use profiling, separate load generator and target system
Scripts not reflecting realistic user behavior	Low validity of results	Validate scenarios with business logic early
Poor AI model performance	False anomalies	Supplement with statistical baselines, tune parameters
Delay in Phase 2 execution	Time pressure in Phase 3	Finalize scenarios early, assign tasks in parallel

1.2.8 Document Structure

1. **Chapter 1 – Introduction**: Context, objectives, scope, methodology.
2. **Chapter 2 – Theoretical Foundations**: Performance Testing concepts, AI support areas, tool comparison.
3. **Chapter 3 – Apache JMeter**: Overview, installation, architecture & components, basic exercises.
4. **Chapter 4 – Experimental Demonstration (JPetStore)**: Scenarios, test data & parameters, execution results, AI-assisted analysis.

Chapter 2

Theoretical Foundations

2.1 Performance Testing Overview

2.1.1 Non-functional Testing

Non-Functional Testing is the process of evaluating aspects of the software that are not related to its functionality. Instead of focusing on testing specific features, it concentrates on factors such as performance, security, reliability, user interaction, and compatibility. The goal of non-functional testing is to ensure that the software performs well not only from a functional perspective but also from other aspects.

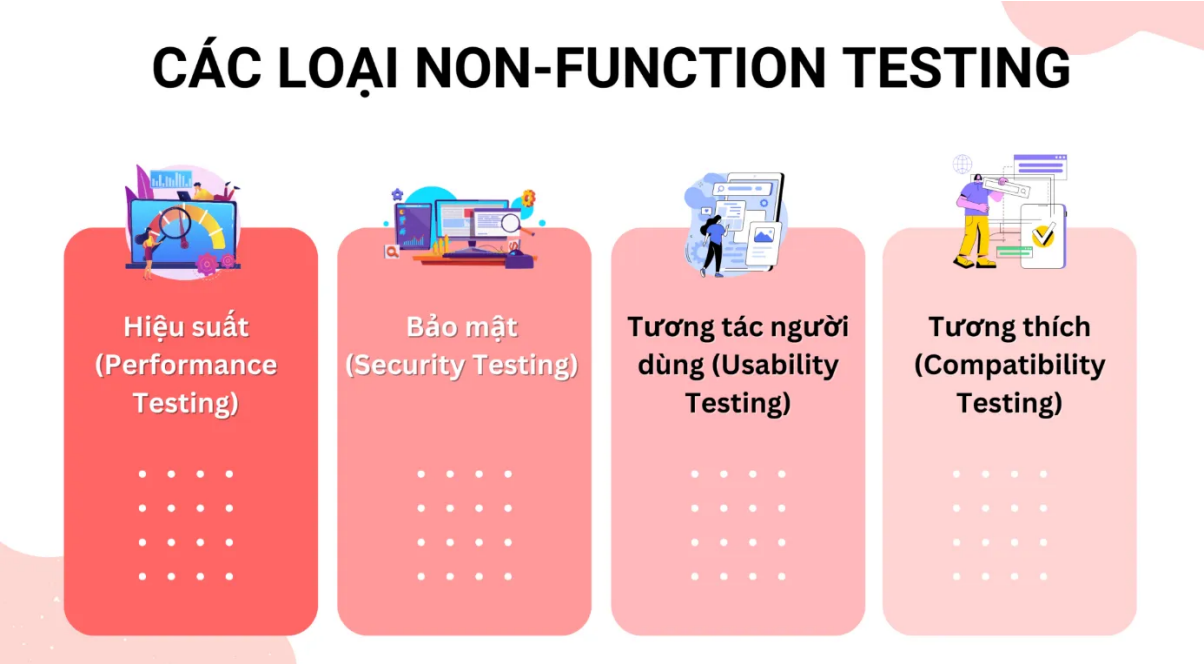


Figure 2.1. Types of non-functional testing.

2.1.2 What is Performance Testing?

Performance Testing is the process of checking whether a software program runs fast, responds quickly, handles load well, how it uses resources (CPU, RAM, bandwidth, etc.), and whether it can scale to accommodate many concurrent users.

Simply put, the goal of performance testing is to detect and eliminate points that cause the software to run slow, become congested, or fail to meet desired performance levels. Performance

testing focuses on the **speed and stability** of the software in real-world use, rather than testing the correctness of its functionality.

The focus of **Performance Testing** is to check for issues in the software program such as:

- **Speed** - Determines if the application responds quickly.
- **Scalability** - Determines the maximum user load the software application can handle.
- **Stability** - Determines if the application is stable under varying loads.

Performance Testing is often used to help identify bottlenecks in a system, establish a baseline for future testing, support effective performance tuning, determine compliance with performance goals and requirements, and gather other relevant operational data to help stakeholders make decisions related to the overall quality of the applications being tested. Additionally, the results from performance testing and analysis can help you estimate the hardware configuration needed to support the applications when you release the product for widespread use.

2.1.3 Types of Performance Testing

Load testing

Load testing is performed to evaluate the system's behavior under increasing load (number of concurrent users/CCU, number of transactions/requests, etc.), to **determine the maximum load (threshold)** that the system can handle.

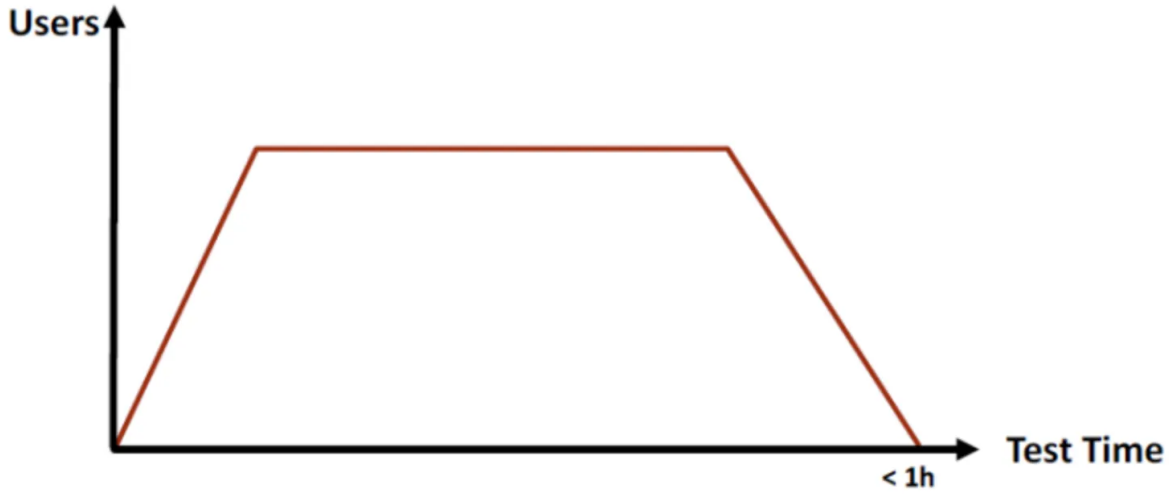


Figure 2.2. Load Testing

A tip to find the threshold quickly is to double the number of concurrent requests sent to the system with each test. For example:

- Test 1: Send 3000 concurrent requests
- Test 2: Send 6000 concurrent requests
- Test 3: Send 12000 concurrent requests

If the system becomes overloaded in Test 3, then in Test 4, reduce the number of requests added in Test 3 by half, meaning:

- Test 4: Send 9000 concurrent requests

Stress testing

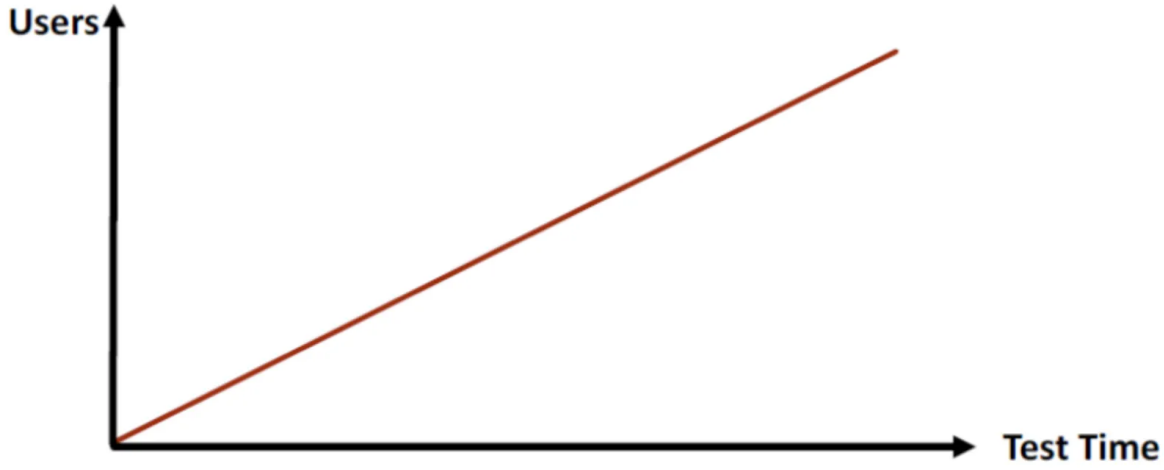


Figure 2.3. Stress Testing

Stress testing is performed to evaluate the system's behavior when the load exceeds the threshold, to find the breaking point and assess the system's ability to recover.

Spike Testing

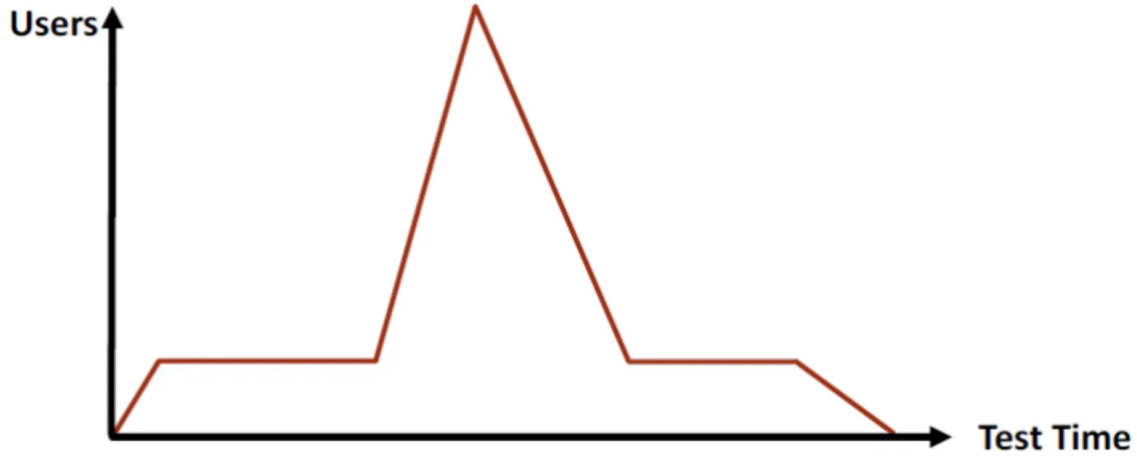


Figure 2.4. Spike Testing

Spike testing is performed to evaluate the system's behavior when the load **suddenly increases for a short period**. Some real-world scenarios that require spike testing:

- E-commerce platforms like Shopee, Lazada, etc., have many large flash sales on holidays and special days.
- Course registration systems for university students.
- Systems for selling football tickets, concert tickets, etc.
- Live streaming systems.

Endurance Testing (Soak Testing)

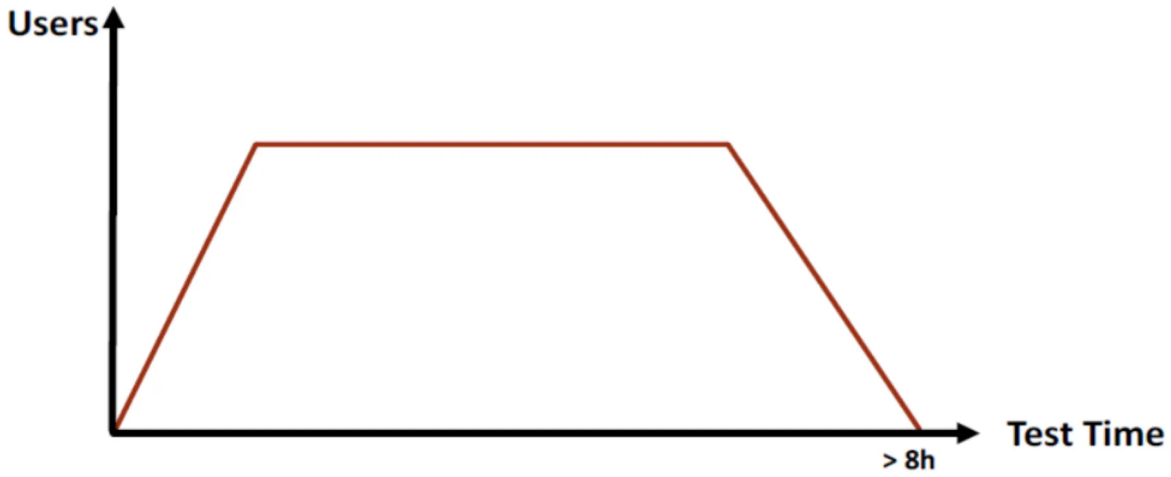


Figure 2.5. Soak Testing

Endurance testing is performed over a long period to evaluate the system's resource usage (memory). Typically, it runs at 70-80% of the system's threshold for more than 8 hours.

Scalability Testing

Scalability testing (or Capacity testing) is a type of testing that aims to evaluate the application's ability to handle load as the number of users or transactions increases. This type of testing helps collect the necessary metrics to forecast and determine the appropriate hardware configuration, ensuring the system can meet larger usage demands in the future.

The main purpose of this testing is to support **system sizing** and to check the application's scalability as its scale increases.

Scalability testing can be considered the **next step** after load testing when a business needs to prepare for growth and expand its user base in the future.

Volume Testing

Volume testing is a type of performance testing that aims to evaluate the software or application's ability to handle a **very large amount of data** in the database.

The main goal is to check if the system remains stable, fast, and error-free when accessing, processing, reporting, or synchronizing with **huge amounts of data**.

- **Example:** Testing the report generation feature when the database has millions of records, or checking the data synchronization speed between large systems.

2.1.4 Common System Performance Issues

Most performance issues revolve around speed, response time, load time, and poor scalability. Speed is often one of the most important attributes of an application. A slow application wastes time, reduces user satisfaction with the system, and can lead to the loss of potential users. Performance testing is done to ensure the application runs fast enough to attract and maintain user interest and satisfaction.

Below is a list of some common performance issues, which also highlights that speed is the most common factor:

- ***Long load time*:** Load time is typically the initial time it takes for an application to launch. This should generally be kept to a minimum. While some applications cannot load in under a minute, a load time of a few seconds is ideal.
- ***Slow response time*:** Response time is the time it takes from when a user inputs data into the application until the application provides a response to that input. Generally, this should be very fast. Again, if users have to wait too long, they will lose interest.
- ***Poor scalability*:** A software product has poor scalability if it cannot handle the expected number of users or if it does not meet the needs of a sufficient range of users. Load testing must be performed to ensure the application can handle the anticipated number of users.
- ***Bottlenecks*:** These are obstacles in the system that degrade the overall system performance. A bottleneck occurs when coding errors or hardware issues cause a drop in throughput under a certain load. Bottlenecks are often caused by a faulty piece of code. The key to fixing the problem is to perform bottleneck testing to find the section of code causing the slowdown and find a solution. Some common performance bottlenecks are: CPU, memory, network, operating system, and hard drive.

2.1.5 Common Metrics in Performance Testing

Depending on the scope of the test and the type of system being tested *(e.g., a web application or a mobile application)*, the measurement parameters or information to be collected and monitored will vary. Below are the general parameters/information often collected and monitored during performance testing:

- **Processor usage** – The amount of time the processor (CPU) spends executing processing threads.
- **Memory usage** – The amount of temporary memory (RAM) used to execute requests. This helps evaluate system performance for optimization.
- **Disk I/O** – The amount of time the disk is busy performing a read or write request.
- **Disk queue length** – The average number of read and write requests queued for the selected disk over a period of time.
- **Bandwidth** – Shows the number of bits per second (bits/s – representing network speed) used during the test.
- **Network output queue length** – The length of the output queue of packets. A queue length greater than one indicates delays and congestion.
- **Response time** – The time from when a user makes a request until the first response is received from the system (server).
- **Throughput** – Measures the number of requests per unit of time, usually seconds. Represents the **total number of requests/unit of time** over a period.
- **Hits per second** – The number of hits/requests sent to the server each second. Often measured during load testing.
- **Thread counts** – The number of currently running and active threads indicates the "health" of the system.

2.2 Performance Testing Tools: Analysis and Comparison

2.2.1 Performance Testing Tools in Practice

- **JMeter**: JMeter is an open-source performance testing tool developed by Apache. It is primarily used to test the durability, performance, and load capacity of web applications, APIs, and servers. JMeter is open-source software written in Java. It was originally created by Stefano Mazzocchi and later redesigned by Apache to improve its graphical user interface (GUI) and add functional testing capabilities. The tool supports many protocols like HTTP, FTP, JDBC, and SOAP, and can be extended with plugins. JMeter is popular in the community for its flexibility and its ability to simulate thousands of virtual users.

Figure 2.6. JMeter Logo

- **k6**: k6 is an open-source performance testing tool built with Go, using JavaScript for writing test scripts. It focuses on simplifying the testing process for developers, offering a command-line interface (CLI) and easy integration into CI/CD pipelines. The tool is centered on load testing for APIs, microservices, and web applications, with advantages like being lightweight, fast, and resource-efficient. k6 stands out with its strong developer community and detailed documentation, making it suitable for projects that require automation and scalability.

Figure 2.7. k6 Logo

- **LoadRunner**: LoadRunner is a commercial performance testing tool developed by Micro Focus. It is designed to simulate thousands of virtual users to evaluate the performance

and load capacity of applications and systems. The tool supports many protocols like HTTP, Web Services, SAP, Oracle, and Citrix, and provides detailed test script recording and playback. LoadRunner is known for its advanced analysis features and integration with project management tools, but it requires licensing fees and significant hardware resources. It is suitable for large enterprises that need large-scale testing and in-depth analysis.

Figure 2.8. LoadRunner Logo

- **NeoLoad**: NeoLoad is a commercial performance testing tool developed by Neotys. It focuses on simulating large loads to assess the performance of web applications, APIs, and mobile apps. With an intuitive GUI, NeoLoad makes it easy to record and design test scripts and offers cloud-based load distribution. The tool stands out for its strong CI/CD integration and detailed reporting but requires licensing fees. NeoLoad is suitable for medium to large businesses needing performance testing, especially in DevOps environments.

Figure 2.9. NeoLoad Logo

2.2.2 Detailed Comparison

Criteria	JMeter	k6	LoadRunner	NeoLoad
Tool Type	Open-source, GUI-based	Open-source, CLI-based	Commercial, GUI & script-based	Commercial, GUI-based

Criteria	JMeter	k6	LoadRunner	NeoLoad
Performance	Resource-intensive, but can be optimized with plugins and distributed load	High performance, optimized for large loads	High performance, but requires powerful hardware	Good performance, optimized for distributed load
Protocol Support	Diverse (HTTP, FTP, JDBC, SOAP, etc.), excellent support	Mainly HTTP, WebSocket, gRPC	Wide (HTTP, Web Services, SAP, Oracle, Citrix), enterprise-focused	Wide support (HTTP, SOAP, REST, etc.), mainly for web and APIs
Ease of Use	Easy with GUI, good for beginners	Easy for those who know JavaScript, difficult for beginners	Difficult for beginners, requires in-depth learning	Has a GUI, but users need to learn how to use it
DevOps Integration	Good integration but requires manual configuration	Easy integration with CI/CD	Strong enterprise integration, but complex	Good CI/CD integration, user-friendly
Reporting	Built-in, detailed via GUI and plugins	Customizable via CLI or k6 Cloud	In-depth reports but paid	Detailed reports, integrates with analysis tools
Community & Plugins	Large, many diverse plugins, strong support	Smaller, fewer plugins	Limited community, relies on Micro Focus support	Moderate community, relies on Neotys support
Cost	Completely free, unlimited	Free (local), paid for k6 Cloud	High cost, requires enterprise license	Paid

→ **Why we chose JMeter**: JMeter stands out for its excellent flexibility with multi-protocol support, a beginner-friendly GUI, and a large open-source community with many free plugins. JMeter offers a free and easily scalable solution through distributed loading, making it superior for projects with limited budgets or high flexibility requirements. Although it has a slight performance disadvantage compared to other tools, JMeter compensates with its detailed configuration options and diverse support, making it suitable for both beginners and experts.

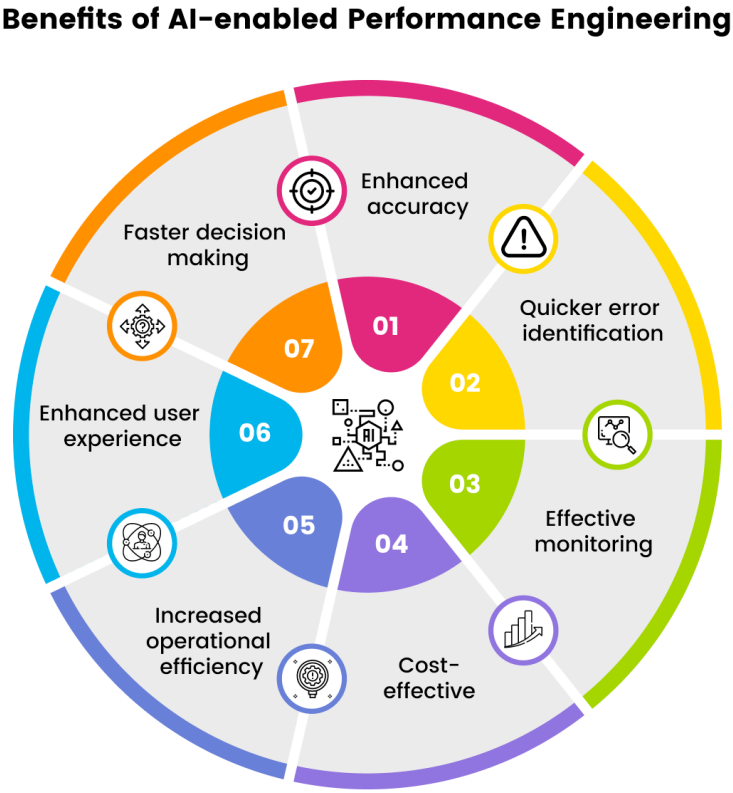
2.3 AI in Performance Testing

2.3.1 Background

When conducting performance tests, one of the most challenging tasks is validating multiple performance indicators such as response time, throughput, and resource utilization. This process is often complex, time-consuming, and heavily dependent on manual effort.

The adoption of Artificial Intelligence (AI) in performance testing aims to overcome these limitations. By automatically analyzing large datasets of performance metrics, AI can identify traffic patterns, detect anomalies, and even provide real-time recommendations. This reduces manual work, shortens analysis time, and improves the accuracy of bottleneck detection.

2.3.2 What is AI in Performance Testing?



At its core, AI in performance testing refers to applying intelligent algorithms and machine learning techniques to make the testing process more efficient and insightful. Instead of relying solely on manual test execution and analysis, AI-driven systems:

- Continuously analyze large volumes of logs, traces, and metrics.
- Automatically detect anomalies and root causes.
- Generate or optimize test scenarios based on real-world traffic data.

This integration transforms performance testing from a manual and reactive process into a proactive and intelligent practice.

2.3.3 Why Use AI in Performance Testing?

- **Smart Resource Management**: Continuously optimizes infrastructure usage to avoid over/under-provisioning.

  - **Example**: When CPU stays above 80% for 5 minutes, AI recommends scaling one more application pod, cutting monthly costs by 35%.

- **Rapid Bottleneck Detection**: Correlates metrics, logs, and traces to isolate slow components fast.
  - **Example**: Flags a specific SQL query adding 450 ms latency to a checkout API within 2 minutes of test start.
- **Predictive Performance (ML Forecasting)**: Uses historical load to predict future demand and capacity needs.
  - **Example**: Forecasts 1,200 concurrent users for a campaign window and advises doubling read replicas 6 hours in advance.
- **Learning from Historical Patterns**: Mines past incidents to prevent repeat failures and generate regression scenarios.
  - **Example**: Detects a recurring memory leak pattern and auto-creates a targeted soak test for the next cycle.
- **Dynamic Alerts & Early Warnings**: Adjusts thresholds based on baselines to reduce noise and catch true anomalies.
  - **Example**: Triggers an alert when response time rises 40% over last week's median instead of a static 2 s rule.
- **Real-Time Monitoring & Auto-Remediation**: Detects degradation and executes predefined corrective actions.
  - **Example**: Automatically restarts a degraded container and shifts 15% traffic to a healthy region.
- **Automated Test Generation**: Derives realistic workloads from production behavior to expand coverage.
  - **Example**: Generates 300 user journeys (e.g., browse → compare → add to cart) from real clickstream data for load rehearsal.

2.3.4 Popular Tools with AI Features

Figure 2.10. Key Performance Testing Tools.

Tool	Description	Key Weakness
Dynatrace	Provides automatic anomaly detection and root cause analysis using Davis AI on logs, traces, and metrics.	High cost; complex installation and configuration.
BlazeMeter	Supports large-scale load and stress testing; includes data analysis and recommendations for script/test data optimization.	Requires technical expertise for effective usage.
LambdaTest (HyperExecute)	Enables cloud-based load tests (e.g., JMeter integration), collects KPIs like response time and error insights across multiple regions.	Lacks advanced AI-powered root cause analysis.
Postman	Provides API performance testing with virtual users, monitoring response time, error rate, and throughput.	Limited scalability for very large or complex performance scenarios compared to specialized tools.

2.3.5 Tool Comparison and Suitability

Rank	Tool	Strengths	Weaknesses	Pricing	Best Fit
1	**Dynatrace**	Comprehensive AI-based analysis, real-time monitoring	Expensive, complex setup	Enterprise-level pricing	Large, complex systems
2	**BlazeMeter**	Powerful load testing, CI/CD integration	Requires technical expertise	From $99/month	DevOps and continuous testing
3	**LambdaTest**	Cloud-based execution, supports multiple regions	Limited AI analysis	From $15/month	Cross-browser and distributed load testing
4	**Postman**	Easy-to-use API performance testing, free tier available	Not suitable for very large-scale tests	Freemium	Small to medium projects, API-focused testing

2.3.6 Current AI Trends in Performance Testing

- **Predictive Performance Analytics**: Forecasts future load, saturation points, and capacity needs instead of only reporting past results.
- **Deep DevOps / CI/CD Embedding**: AI engines run inside pipelines to gate merges with latency, error, and regression signals.
- **Self-Healing Test Assets**: Automatically updates locators, payloads, or workflow steps when UI/API contracts drift.
- **Automated Capacity & Cost Optimization**: Trains on historical usage to recommend right-sizing, scaling windows, and instance families.
- **Autonomous Monitoring & Remediation**: Progressing from anomaly detection to triggering rollbacks, scaling actions, or traffic re-routing.
- **Context-Aware Alerting**: Dynamic baselines reduce noise by comparing against seasonality, release windows, and user cohorts.
- **Root Cause Correlation Graphs**: Causal mapping links spikes in latency to specific services, queries, or infrastructure layers.
- **Synthetic + Real User Blending**: AI fuses RUM, APM, and synthetic journeys to generate more realistic load models.
- **Intelligent Test Scenario Generation**: Derives high-impact user flows from production clickstream clustering.

- **Explainability & Trust Tooling**: Emerging features expose why an anomaly was flagged (metric deltas, dependency path, confidence score).

2.3.7 Challenges and Limitations

- **Bias & Prompt Design Errors**: Inaccurate or incomplete prompts can yield unrealistic test cases or costly recommendations.
- **High Initial Investment**: Significant upfront spend on infrastructure, licensing, integration, and specialized expertise.
- **Vendor Lock-In**: Migrating between platforms (e.g., Dynatrace to BlazeMeter) is complex, risky, and expensive.
- **Black-Box Nature**: Limited explainability of AI outputs lowers tester trust and slows adoption.
- **Data Dependency**: Insufficient or poor-quality historical data degrades model accuracy and prediction reliability.
- **Learning Curve**: Teams require training time to interpret insights and tune models effectively.
- **Over-Reliance on AI**: Excess dependence can reduce manual skill development and critical performance analysis.

Chapter 3

Apache JMeter

This section introduces Apache JMeter—identified in the prior comparison table as a leading open-source tool for performance testing. It will move from core concepts to practical details and conclude with an end-to-end performance test scenario built in JMeter.

3.1 JMeter's Overall Architecture

Apache JMeter is an open-source tool designed for performance and load testing of web applications and various other services. JMeter has a modular, plugin-based architecture that helps organize and simulate test flows.

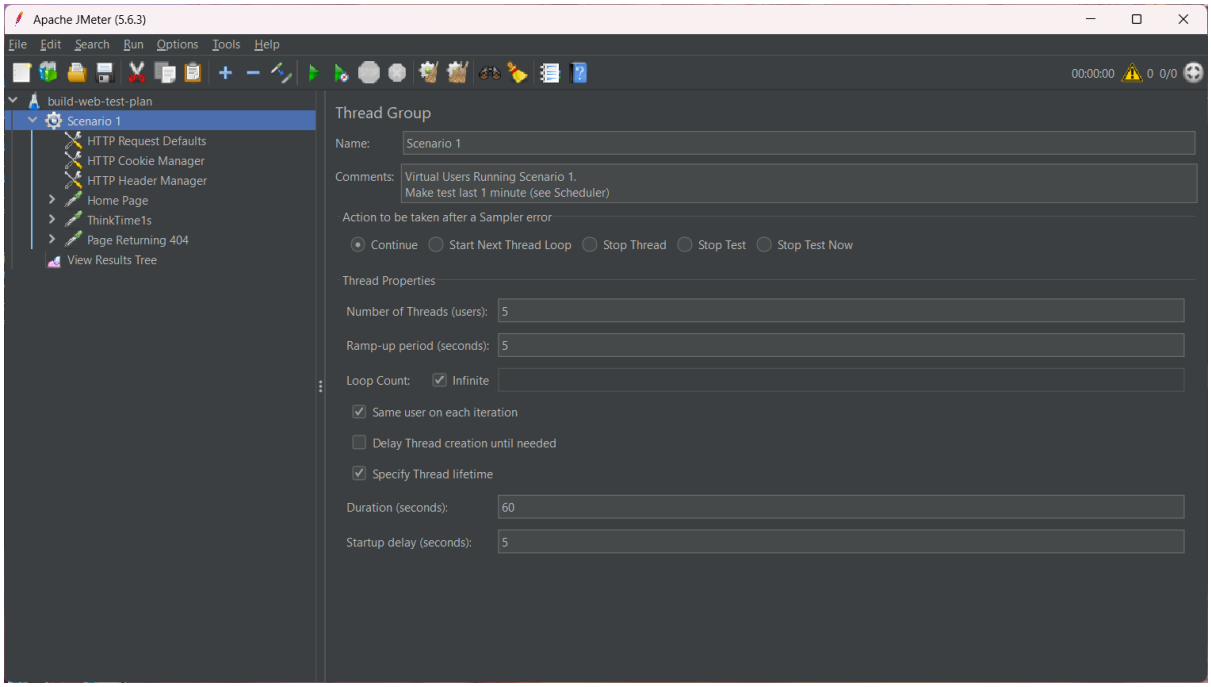


Figure 3.1. JMeter GUI

The main components in JMeter's architecture include:

- **Test Plan**: The root structure of every test, defining the entire test configuration and behavior.
- **Thread Group**: Represents virtual users, simulating a large number of users accessing the system.
- **Samplers**: Send specific requests to the system under test.

- **Logic Controllers**: Control the execution flow within a Thread Group.
- **Listeners**: Record and display test results.
- **Configuration Elements**: Provide default settings for Samplers.
- **Timers**: Add delays between requests.
- **Assertions**: Verify that the response matches expectations.
- **Pre/Post-Processors**: Process data before or after a request is sent.

These components can be nested in a hierarchical structure within a Test Plan to create complex test scenarios.

3.2 Thread Group

A Thread Group is the primary element for configuring how users are simulated. Each Thread Group can be configured with the following settings:

- **Number of Threads (users)**: The number of virtual users to simulate.
- **Ramp-Up Period (seconds)**: The time it takes to start all the virtual users.
- **Loop Count**: The number of times each user will repeat the test script.

Thread Groups also allow you to configure what happens on an error (e.g., stop the test, skip the sampler) and can contain many child elements like Samplers, Timers, and Assertions.

For example, if you configure 50 threads with a 10-second ramp-up and a loop count of 2, JMeter will start 5 threads every second, and each user will send their requests twice.

3.3 Sampler

A Sampler is responsible for sending different types of requests to the system being tested. Each Sampler corresponds to a specific protocol or action.

Some common types of Samplers include:

- **HTTP Request**: Sends HTTP/HTTPS requests to a web server.
- **JDBC Request**: Sends SQL queries to a database via JDBC.
- **FTP Request**: Sends requests to upload or download files from an FTP server.
- **SOAP/XML-RPC Request**: Sends requests to SOAP-based web services.
- **Java Request**: Allows you to directly call custom Java classes.
- **JMS Request**: Used to send/receive messages through a JMS system.

Samplers are crucial as they determine what system is being tested and how.

3.4 Listener

A Listener is a component that helps collect, record, and display test results. Listeners can write to a log, display real-time reports, or save results to a file for later analysis.

Some typical Listeners are:

- **View Results Tree**: Shows the details of every single request and response.
- **Summary Report**: Provides aggregate statistics like the number of requests, average response time, error rate, etc.
- **Aggregate Report**: Displays advanced aggregate information like standard deviation, response time percentiles, etc.
- **Graph Results**: Plots test data on a graph.

- **View Results in Table**: Displays a list of requests in a table format.

Choosing the right Listener makes analyzing system performance more intuitive and accurate.

3.5 Timers, Assertions, Pre-Processors, and Post-Processors

1. **Timers**

Timers are used to add delays between requests to simulate realistic user behavior. Some commonly used Timers are:

- **Constant Timer**: Adds a fixed delay.
- **Uniform Random Timer**: Adds a random delay with a uniform distribution.
- **Gaussian Random Timer**: Adds a random delay with a normal (Gaussian) distribution.
- **Synchronizing Timer**: Pauses threads to send requests all at once.

2. **Assertions**

Assertions are used to verify that the response from the server is correct and valid. Some common Assertions include:

- **Response Assertion**: Checks the response content or status code.
- **Duration Assertion**: Checks that the response time does not exceed a specified value.
- **Size Assertion**: Checks the size of the response.
- **JSON/XML Assertion**: Checks the structure and data of JSON/XML in the response.

Assertions help evaluate the correctness of the system, not just its performance.

3. **Pre-Processors**

A Pre-Processor is executed before a Sampler runs. It is often used to:

- Prepare input data.
- Generate dynamic tokens, headers, or values.
- Call functions from BeanShell or JSR223 for custom logic.

An example is "User Defined Variables," which sets up values to be used in subsequent requests.

4. **Post-Processors**

A Post-Processor is executed after a Sampler sends a request. It is used to:

- Extract data from the response to use in a later step.
- Save data, write to a log, or process the response string.
- The most popular are the **Regular Expression Extractor**, **JSON Extractor**, and **XPath Extractor**.

3.6 Installing Apache JMeter: Step-by-Step Guide

1. **Step 1: Install Java** JMeter requires Java (JDK) 8 or later (Java 17 is a well-tested choice). Supported operating systems: Windows, Linux, macOS. No extra runtime needed beyond Java. Download and install one of:

   - Oracle JDK: https://www.oracle.com/java/technologies/javase-downloads.html
   - OpenJDK (Adoptium): https://adoptium.net Verify installation:

```
java -version
```

2. **Step 2: Download JMeter** Get the latest binary (.zip for Windows, .tgz for Linux/macOS): https://jmeter.apache.org/download_jmeter.cgi

Figure 3.2. Apache JMeter Download Screen

3. **Step 3: Extract**

   - Download the archive.
   - Extract to any folder (no installer required).
   - On Windows you will use the .bat launcher; on macOS/Linux the shell script in bin/.

4. **Step 4: Launch** In the extracted bin directory run:

   - Windows:

```
jmeter.bat
```

   - macOS / Linux:

```
./jmeter
```

The JMeter GUI will open.

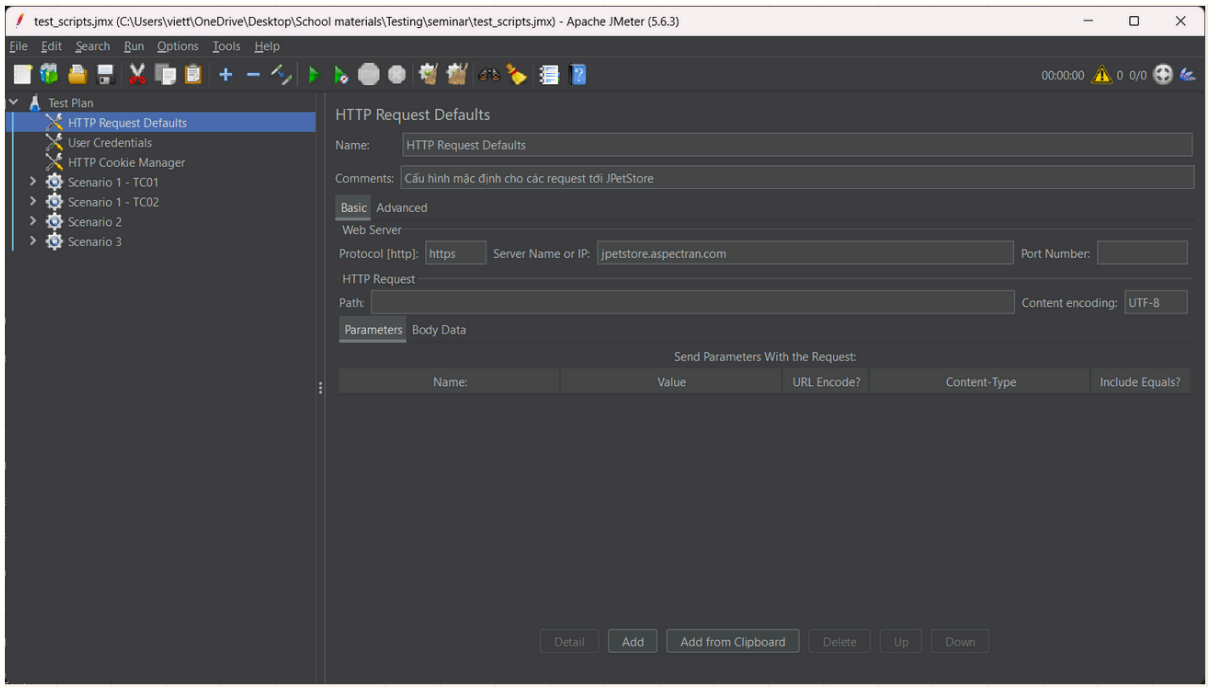


Figure 3.3. JMeter main screen

Chapter 4

Experimental Demonstration

4.1 Hands-On Performance Testing Scenario with JMeter

4.1.1 Overview

The purpose of this Performance Testing Plan is to outline the strategy for planning and managing performance testing activities for the JPetStore system, an open-source web application simulating an e-commerce process. This document details the **Performance Testing Approach** using Apache JMeter and serves as a comprehensive guide for the team's testing process.

4.1.2 Objectives

- Identify key functionalities and business flows for performance testing.
- Define the testing approach, including JMeter usage, input data, and evaluation criteria.
- Specify required resources: tools, test environments, data, and personnel.
- List deliverables: test scripts (`.jmx`), execution results (`.csv`), statistical charts, reports, and demo videos.

4.1.3 Scope

The project involves both research and practical performance testing for the JPetStore system, including:

- Studying performance testing theory: concepts, processes, and its role in software testing.
- Researching types of performance testing: Load, Stress, Spike, and Endurance Testing.
- Comparing performance testing tools: Apache JMeter, NeoLoad, WebLOAD, LoadUI, and LoadRunner.
- Deep exploration of Apache JMeter: architecture, components, installation, and configuration.
- Investigating AI applications in performance testing, such as automated analysis, error prediction, and process optimization.
- Conducting tests on the **JPetStore** demo system.
- Designing and executing test scenarios for common user flows:
  - **Scenario 1**: Browsing products without logging in.
  - **Scenario 2**: Logging in and completing a full purchase process.
  - **Scenario 3**: Logging in and accessing personal account information.

4.1.4 Testing Activities

The team will conduct system-level performance testing to evaluate the response time and stability of JPetStore under concurrent user access. Key activities include: - Designing and

executing **test scenarios** to simulate realistic user behaviors for the three specified flows: - **Scenario 1**: Browsing products without logging in. - **Scenario 2**: Logging in and completing a full purchase. - **Scenario 3**: Logging in and accessing account information. - Using **Apache JMeter** to create and execute performance test scenarios. - Collecting and analyzing performance metrics: - **Response Time** - **Throughput** - **Error Rate** - Applying techniques such as: - Dynamic input data (CSV). - Timers to simulate think time. - Assertions to validate response results.

4.1.5 Deliverables

- Test scripts (`.jmx` files).
- Execution results (`.csv` files).
- Statistical charts and reports.
- Demo videos showcasing the testing process.

4.2 Scenario 1 – Browse-Only Load

4.2.1 Purpose

Evaluate system performance for users browsing products without logging in, simulating real-world catalog and product detail access. Measure **response time**, **error rate**, and **throughput** under typical load.

4.2.2 User Flow

Step	Action	HTTP Method	Think Time
1	Access homepage (`/`)	`GET`	2–5s
2	Access category (e.g., `/categories/FISH`)	`GET`	3–6s
3	Select product (e.g., `/products/FI-SW-01`)	`GET`	5–10s
4	View item details (e.g., `/products/FI-SW-01/items/EST-1`)	`GET`	—

4.2.3 Test Case 1 – Load Testing

General Information

Attribute	Details
Scenario Name	Browse-Only Load
Test Case ID	TC01_BrowseOnly_Load
Objective	Test system performance for browsing without login
Test Type	Load Testing
Components	Homepage, category, product, item details
Input Data	CSV file: `categoryId`, `productId`, `itemId`

Attribute	Details
Sample Data	FISH, FI-SW-01, EST-1
Pre-condition	JPetStore running at https://jpetstore.aspectran.com

JMeter Configuration

Parameter	Value
Threads	40
Ramp-up Time	30s
Loop	Infinite
Duration	5 min

JMeter Setup

- **Data Source**: CSV (`product_summary.csv`) with `categoryId`, `productId`, `itemId`.
- **Logic Controller**:
  - Random Controller: Selects random CSV row.
  - If Controller: Ensures valid `categoryId`, `productId`, `itemId`.
- **Samplers**: 4 HTTP Requests using dynamic endpoints (`${categoryId}`, `${productId}`, `${itemId}`).
- **Timers**: Uniform Random Timer (2000–5000ms, 3000–6000ms, 5000–10000ms).
- **Assertions**: Response Code = 200; Response Body contains "Product ID" or "Add to Cart"/"Item ID".

Measurement Goals

- 95% response time $<$ 2000ms.
- Error Rate = 0%.
- Stable throughput over 5 minutes.

4.2.4 Test Case 2 – Spike Testing

General Information

Attribute	Details
Scenario Name	Spike Load with Recovery
Test Case ID	TC02_Spike_Load
Objective	Assess performance and recovery under sudden load spikes
Test Type	Spike Testing
Components	Homepage, category, product, item details
Input Data	CSV file: `categoryId`, `productId`, `itemId`
Sample Data	FISH, FI-FW-01, EST-1
Pre-condition	JPetStore running at https://jpetstore.aspectran.com

JMeter Configuration (Ultimate Thread Group)

Stage	Threads	Initial Delay (s)	Startup Time (s)	Hold Load (s)	Shutdown Time (s)	Description
1	10	0	5	60	10	Warm-up
2	25	65	10	60	0	Gradual increase
3	70	135	0	20	5	Spike 1
4	25	155	5	60	0	Recovery
5	100	220	0	20	5	Spike 2
6	25	240	5	60	5	Stabilize

- **Total Duration**: ≈ 5 min.

JMeter Setup

- **Data Source**: CSV (`product_summary.csv`) with `categoryId`, `productId`, `itemId`.
- **Logic Controller**: Random Controller; If Controller for valid data; optional JSR223 Sampler (Groovy).
- **Samplers**: 4 HTTP Requests with dynamic endpoints.
- **Timers**: Uniform Random Timer (2000–5000ms, 3000–6000ms, 5000–10000ms).
- **Assertions**: Response Code = 200; Response Body contains "Product ID" or "Add to Cart"/"Item ID".

Measurement Goals

- 95% response time < 2000ms (even during spikes).
- Error Rate 3% during spikes.
- Throughput spikes and stabilizes within 1–2 min.

4.3 Scenario 2 – Full Purchase Flow Under Load

4.3.1 Purpose

Simulate a complete purchase process (login, category/product selection, cart addition, checkout). Assess system performance under complex, session-based operations. Measure **response time**, **error rate**, and **throughput** under sustained load.

4.3.2 User Flow

Step	Action	HTTP Method	Think Time
1	Access homepage (`/`)	`GET`	2–4s
2	Access login page (`/account/signonForm`)	`GET`	3–5s
3	Login (`/account/signon`)	`POST`	3–5s
4	Access category (`/categories/{categoryId}`)	`GET`	3–6s

Step	Action	HTTP Method	Think Time
5	Access product (`/products/{productId}`)	`GET`	2–4s
6	Access item (`/products/{productId}/items/{itemId}`)	`GET`	5–8s
7	Add to cart (`/cart/addItemToCart?itemId=...`)	`GET`	0
8	View cart (`/cart/viewCart`)	`GET`	2–4s
9	Access order form (`/order/newOrderForm`)	`GET`	1–3s
10	Submit order (`/order/newOrder`)	`POST`	1–2s
11	Confirm order (`/order/submitOrder`)	`POST`	—

4.3.3 Test Case 1 – Load Testing

General Information

Attribute	Details
Scenario Name	Full Purchase Flow Under Load
Test Case ID	TC01_Purchase_Load
Objective	Evaluate system under full purchase flow
Test Type	Load Testing
Components	Homepage, login, category, product, cart, checkout
Input Data	CSV files: `username`, `password`; `categoryId`, `productId`, `itemId`
Sample Data	j2ee, j2ee, FISH, FI-SW-01, EST-1
Pre-condition	JPetStore running at https://jpetstore.aspectran.com

JMeter Configuration

Parameter	Value
Threads	40
Ramp-up Time	45s
Loop	Infinite
Duration	5 min

JMeter Setup

- **Data Sources**:
  - CSV (`users_60.csv`): `username`, `password`.
  - CSV (`product_summary.csv`): `categoryId`, `productId`, `itemId`.

- **Logic Controller**: Thread Group for all steps; Random Controller for product access; If Controller for valid data; optional JSR223 Sampler.
- **Samplers**: HTTP Requests for each step using dynamic variables (`${categoryId}`, `${productId}`, `${itemId}`, `${username}`, `${password}`).
- **Timers**: Uniform Random Timer for user think time.
- **Assertions**: Response Code = 200 (302 for `submitOrder` if redirects disabled); confirm order success in final response.

Measurement Goals

- 95% response time < 3000ms for checkout steps.
- Error Rate 0%.
- Successful order confirmation.
- No timeouts on login, cart, or checkout.

4.4 Scenario 3 – Concurrent Login & Account Access

4.4.1 Purpose

Test system stability under continuous login and account access (endurance test). Evaluate **response time**, **error rate**, **memory**, and **throughput** with frequent session creation/deletion. Analyze error behavior and recovery during load changes.

4.4.2 User Flow

Step	Action	HTTP Method	Think Time
1	Access homepage (`/`)	`GET`	2–3s
2	Access login page (`/account/signonForm`)	`GET`	4–6s
3	Login (`/account/signon`)	`POST`	3–5s
4	Access “My Account” (`/account/editAccountForm`)	`GET`	4–7s
5	Logout (`/account/signoff`)	`GET`	—

4.4.3 Test Case 1 – Endurance Testing

General Information

Attribute	Details
Scenario Name	Concurrent Login & Account Access
Test Case ID	TC01_Concurrent_Login
Objective	Test system under continuous session creation/deletion
Test Type	Endurance Testing
Components	Homepage, login, account, logout
Input Data	CSV file: `username`, `password`

Attribute	Details
Sample Data	j2ee, j2ee

JMeter Configuration (Ultimate Thread Group)

Threads	Initial Delay	Startup Time	Hold Load	Shutdown Time	Notes
5	0s	25s	300s	50s	Warm-up
10	325s	50s	300s	75s	Medium load
25	675s	75s	300s	100s	High load
40	1050s	100s	10800s (3h)	75s	Main endurance
25	11950s	75s	300s	50s	Reduce load
10	12325s	50s	300s	25s	Recovery

JMeter Setup

- **Data Source**: CSV (`users_60.csv`): `username`, `password`.
- **Logic Controller**: Thread Group per load stage; executes login → account → logout.
- **Samplers**: HTTP Requests for each step using `${username}`, `${password}`.
- **Timers**: Uniform Random Timer for user think time.
- **Assertions**: Response Code = 200; verify `editAccountForm` content post-login.

Technical Notes

- Demo runs up to login (`POST /signon`) for first 1.5h.
- Monitor error rate closely from minutes 15–22 (load increases from 5 to 40 users).
- Error rate may spike to $\approx 8\%$ but stabilizes at $\approx 6\%$.
- Ensure stable network for long endurance runs.

Measurement Goals

- 95% response time post-login < 2000ms.
- Average error rate $< 5\%$ during endurance.
- Complete login → account access → logout cycle without errors.
- Track error rate changes during load increase/decrease.

4.5 Execution Results and AI-Supported Analysis

4.5.1 Scenario 1 – Browse-Only Load

Test Case TC01 – Load Testing

Metric	Value
Total Requests	2,902
Avg. Response Time	**118 ms**
95% Line	**244 ms**
Max Response Time	**509 ms**
Error Rate	**0.00%**
Throughput	**9.75 req/sec**

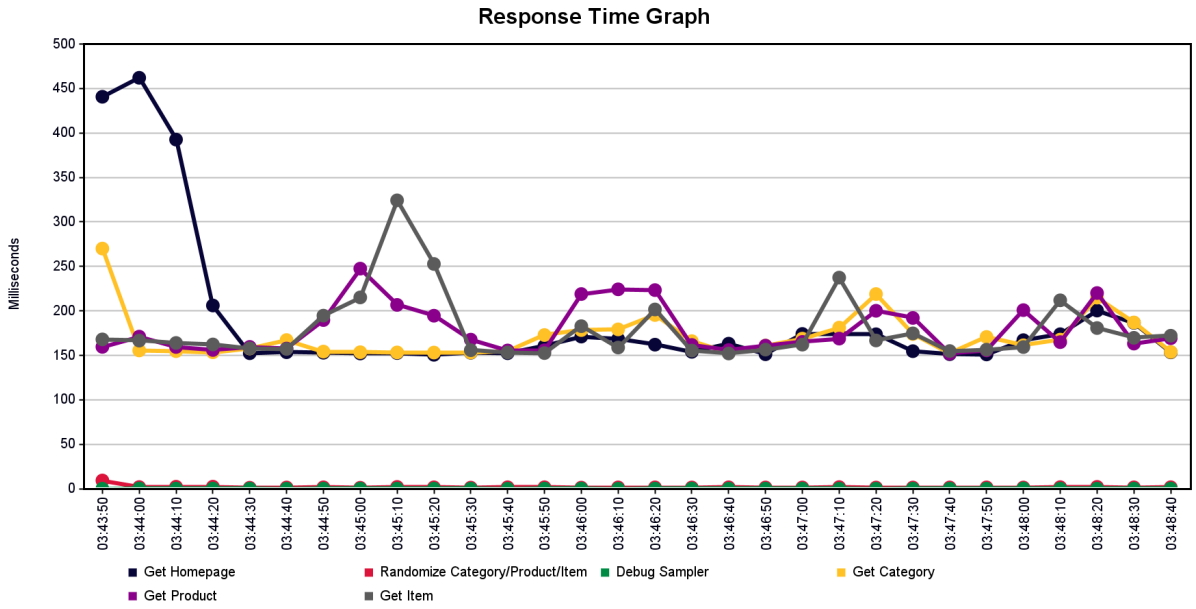


Figure 4.1. Response Time Graph for Test Case 01

$\Longrightarrow$ System handled browsing load smoothly with no errors and low latency. 95% of requests completed in under 244 ms.

Test Case TC02 – Spike Testing

Metric	Value
Total Requests	2,343
Avg. Response Time	**134 ms**
95% Line	**455 ms**
Max Response Time	**509 ms**
Error Rate	**0.00%**
Throughput	**7.67 req/sec**

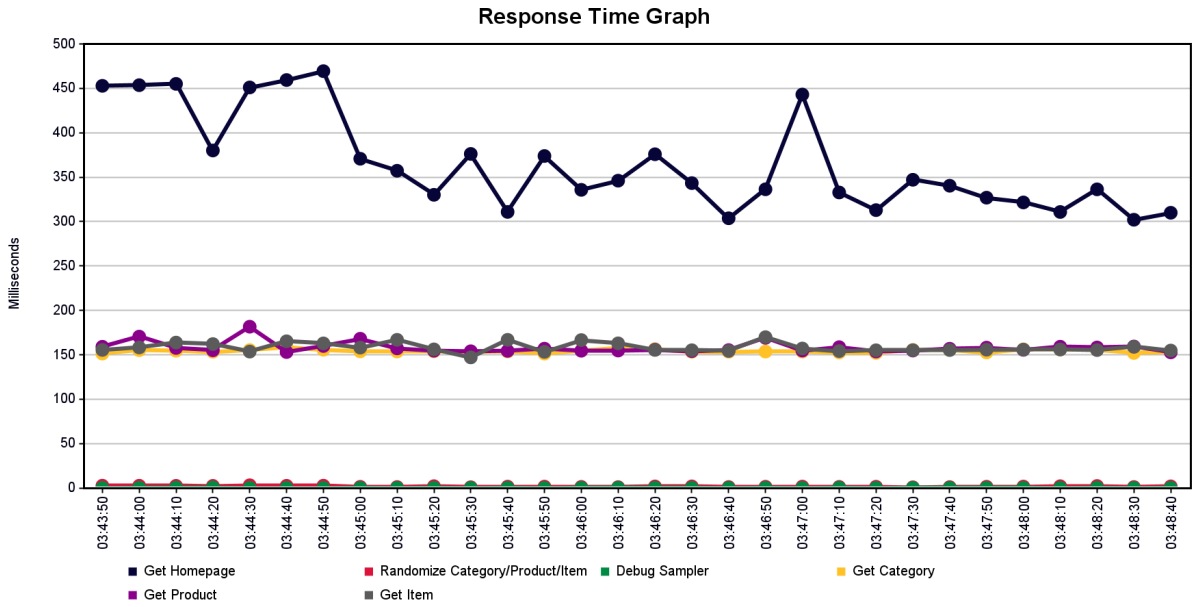


Figure 4.2. Response Time Graph for Test Case 02

$\Longrightarrow$ During sudden traffic spikes, performance stayed stable. Slight increase in 95% percentile observed, but still under 500 ms — well within acceptable thresholds.

4.5.2 Scenario 2 – Full Purchase Flow

Metric	Value
Total Requests	3,651
Avg. Response Time	**195 ms**
95% Line	**316 ms**
Max Response Time	**598 ms**
Error Rate	**0.00%**
Throughput	**12.3 req/sec**

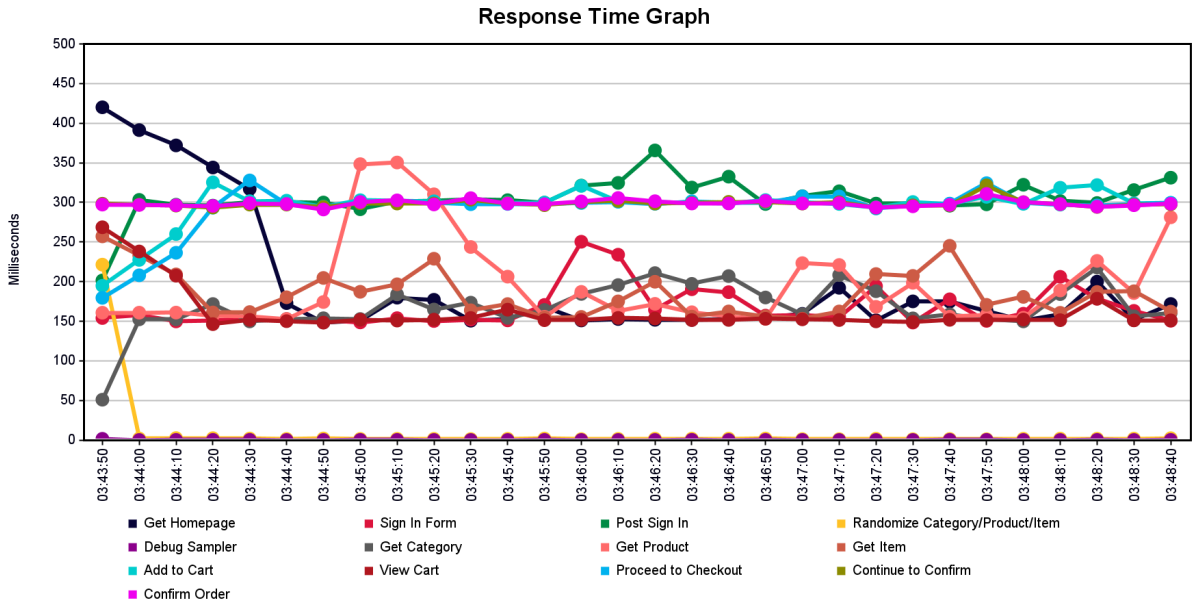


Figure 4.3. Response Time Graph for Scenario 02

$\Longrightarrow$ The system managed the full login-to-checkout journey under consistent load with impressive stability. Checkout steps (Add to Cart $\rightarrow$ Submit Order) had slightly higher latency but remained within acceptable bounds.

4.5.3 Scenario 3 – Concurrent Login & Account Access (Endurance Test)

Metric	Value
Total Requests	**130,575**
Avg. Response Time	**241 ms**
95% Line	**445 ms**
Max Response Time	**31,957 ms** (!)
Error Rate	**0.02%**
Throughput	**10.29 req/sec**

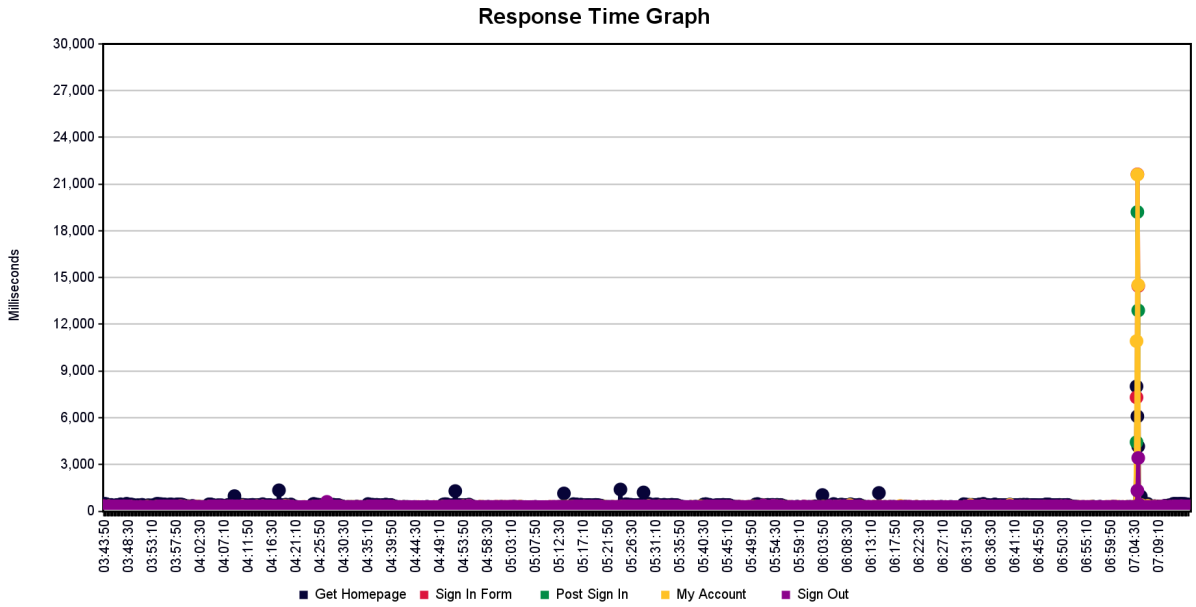


Figure 4.4. Response Time Graph for Scenario 03

$\Longrightarrow$ This was the longest-running and most intense test. While average performance was stable, a few extreme outliers (31s max) were recorded. Error rate remained low ($\approx 0.02\%$) but spiked during ramp-up periods.

4.6 Insights & Analysis

1. Anomaly Detection

   - Observed a small set of extreme latency outliers in Scenario 3 (20–31 s vs 95% $<$ 450 ms).
   - Pattern: isolated, not clustered, suggest long-tail pauses.
   - Likely causes: JVM GC pauses, transient thread pool saturation, external dependency stalls, or network hiccups.
   - Actions:
     - Correlate timestamps with server GC / CPU / heap graphs.
     - Enable server-side slow request logging (include thread dump trigger >5 s).
     - Add PerfMon / JFR listeners for next endurance run.
     - Consider tuning: heap sizing, GC algorithm, connection pool max, async IO.

2. Percentile Drift

- All scenarios: 95th percentile $<$ 500 ms $\rightarrow$ strong interactive responsiveness.
- Scenario 2 checkout steps show relatively higher (still $<$ 600 ms) due to multi-step stateful operations (cart + order submission).
- No widening gap between avg and 95th $\rightarrow$ latency distribution tight except rare Scenario 3 spikes.
- Actions:
  - Track p99 in future runs to confirm tail stability.
  - Segment metrics by step (login, add-to-cart, submit order) to isolate heavier endpoints.

3. Load Resilience

- Scenarios 1 & 2: Throughput stable, zero errors → healthy under typical and spike patterns.
- Scenario 3: Long endurance with gradual ramps surfaced tail spikes without significant error amplification (0.02% overall).
- Recovery after load transitions was fast; no sustained degradation.
- Actions:
  - Introduce controlled chaos (network latency injection) to validate resilience.
  - Add soak-phase leak detection: monitor open sessions, threads, heap growth slope.

4. Resource Efficiency

- Throughput range: 7–12 req/s across mixes; proportional to concurrent user behavior patterns.
- Bandwidth usage stable; no evidence of payload bloat.
- Low error rate implies minimal wasted retries or amplified load.
- Actions:
  - Capture server-side CPU per request to confirm headroom for scaling.
  - Add response size assertions to detect accidental payload inflation in future builds.

5. Prioritized Recommendations

6. Instrument & correlate (GC, thread pools, DB timings) to explain Scenario 3 outliers.

7. Add p99 / p99.9 tracking and step-level dashboards.

8. Implement slow request tracing (>2 s) with contextual metadata.

9. Run repeat endurance test after tuning to validate reduction of long-tail spikes.

10. Expand scenario coverage to include concurrent checkout + browsing overlap.

11. Success Criteria Validation

- Met: 95% response time targets, negligible error rates, stable throughput.
- Partially Met: Tail latency consistency (affected by rare extreme spikes).
- Next Target: Reduce max latency outliers to <5 s and maintain p99 < 1 s.

7. Suggested AI Augmentation (Future)

- Automated anomaly clustering on response time + server metrics.
- Predictive alerting when latency slope indicates impending tail spike.
- Dynamic test load shaping based on real-time percentile degradation.
- **Summary**: Core performance is strong for median and 95th percentiles; focus shifts to eliminating rare but severe tail latency events observed during extended concurrency in Scenario 3.